

QR Code

QR Code

1. Introduction to QR Code
2. Structure of QR Code
3. Features of QR Code
4. QR Code Creation and Recognition

1. Introduction to QR Code

A QR code is a type of two-dimensional barcode. QR comes from the abbreviation of the English "Quick Response", meaning quick response, which originates from the inventor's hope that the QR code can allow its content to be quickly decoded. QR codes not only have large information capacity, high reliability, and low cost, but can also represent multiple types of text information such as Chinese characters and images. They have strong anti-counterfeiting and confidentiality properties and are very convenient to use. More importantly, the QR code technology is open source.

2. Structure of QR Code

Image	Explanation
	Positioning markings indicate the orientation of the QR code.
	Alignment markings If the QR code is very large, these additional elements help with positioning.
	Timing pattern Through these lines, the scanner can identify how large the matrix is.
	Version information This specifies the version number of the QR code being used. Currently, there are 40 different version numbers for QR codes. The version numbers used in the retail industry are usually 1-7.
	Format information The format pattern contains information about the error tolerance and data mask patterns, and makes scanning the code easier.
	Data and error correction keys These patterns store the actual data.
	Quiet zone This area is very important for the scanner. Its function is to separate itself from the surrounding area.

3. Features of QR Code

The data value in a QR code contains repeated information (redundancy). Therefore, even if up to 30% of the QR code structure is damaged, the readability of the QR code is not affected. The storage space of a QR code is up to 7089 bits or 4296 characters, including punctuation marks and special characters, all of which can be written into the QR code. In addition to digits and characters, words and phrases (such as URLs) can also be encoded. As more data is added to the QR code, the code size increases and the code structure becomes more complex.

4. QR Code Creation and Recognition

Source code path: /home/pi/yahboomcar_ws/src/yahboomcar_visual/simple_qrcode

Installation

```
python3 -m pip install qrcode pyzbar
sudo apt-get install libzbar-dev
```

- Creation

Create the qrcode object

```
'''
    Parameter meaning:
    version: an integer with a value of 1~40, controls the size of the QR code
    (the minimum is 1, which is a 12x12 matrix).
        If you want the program to determine it automatically, set the value
    to None and use the fit parameter.
    error_correction: controls the error correction function of the QR code. You
    can use the following 4 constants.
    ERROR_CORRECT_L: about 7% or less errors can be corrected.
    ERROR_CORRECT_M (default): about 15% or less errors can be corrected.
    ERROR_CORRECT_H: about 30% or less errors can be corrected.
    box_size: controls the number of pixels contained in each small grid of the
    QR code.
    border: controls the number of grids contained in the border (the distance
    between the QR code and the image boundary) (the default is 4, which is the
    minimum value specified by the relevant standard)
'''
qr = qrcode.QRCode( version=1,
error_correction=qrcode.constants.ERROR_CORRECT_H, box_size=5, border=4,)
```

Add a logo to the QR code

```
# If the logo path exists, add the logo image
my_file = Path(logo_path)
if my_file.is_file(): img = add_logo(img, logo_path)
```

Note: When using Chinese, Chinese characters need to be added.

Run it directly using python3 + py file, then enter the content to be generated and press Enter to confirm.



- Recognition

```
def decodedisplay(image, font_path):
    gray = cv.cvtColor(image, cv.COLOR_BGR2GRAY)
    # The output Chinese characters need to be converted into Unicode encoding
    form first
    barcodes = pyzbar.decode(gray)
    for barcode in barcodes:
        # Extract the position of the bounding box of the QR code
        (x, y, w, h) = barcode.rect
        # Draw the bounding box of the barcode in the image
        cv.rectangle(image, (x, y), (x + w, y + h), (255, 0, 0), 5)
        encoding = 'UTF-8'
        # To draw it, it first needs to be converted into a string
        barcodeData = barcode.data.decode(encoding)
        barcodeType = barcode.type
        # Draw the data and type on the image
        pilimg = Image.fromarray(image)
        # Create a brush
        draw = ImageDraw.Draw(pilimg)
        # Parameter 1: font file path, Parameter 2: font size
        fontStyle = ImageFont.truetype(font_path, size=12, encoding=encoding)
        # Parameter 1: print coordinates, Parameter 2: text, Parameter 3: font
        color, Parameter 4: font
        draw.text((x, y - 25), str(barcode.data, encoding), fill=(255, 0, 0),
        font=fontStyle)
        # Convert PIL image to cv2 image
        image = cv.cvtColor(np.array(pilimg), cv.COLOR_RGB2BGR)
        # Print the barcode data and barcode type to the terminal
        print("[INFO] Found {} barcode: {}".format(barcodeType, barcodeData))
    return image
```

- Effect demonstration

Run it directly using python3 + py file

(To parse the code, run Qrcode_Parsing_USB.py with a USB camera; run Qrcode_Parsing_CSI.py with a CSI camera)

